Algorithmes de Tri et de Recherche

S3_LIA_IFOAD_2026-2027

Dr Gouayon KOALA

Institut de Formation Ouverte et A Distance

(IFOAD - UJKZ)

gkoala25@gmail.com

Contenu

I. Rappel : les chaînes de caractères
II. Algorithmes de tri stable
III. Algorithmes de tri non stable
IV. Algorithmes de recherche séquentielle
V. Algorithmes de recherche dichotomique
VI. Arbres binaires
VII. Graphes
VIII. Les fichiers

I. Les chaînes de caractères

I. Les Chaînes de Caractères

Définition:

Une **chaîne de caractère** (appelée string en anglais) est une suite de caractères, c'est-à-dire un ensemble de symboles faisant partie du jeu de caractères, défini par le code ASCII.

En langage C, une chaîne de caractères est un tableau, comportant plusieurs données de type char, dont le dernier élément est le caractère nul \0, c'est-à-dire le premier caractère du code ASCII (dont la valeur est 0). Ce caractère est un caractère de contrôle (donc non affichable) qui permet d'indiquer une fin de chaîne de caractères. Ainsi une chaine composée de n éléments sera en fait un tableau de n+1 éléments de type char. On peut par exemple représenter la chaîne "Bonjour" de la manière suivante:



I. Les Chaînes de Caractères

Créer une chaine de caractères

Pour définir une chaîne de caractères en langage C, il suffit de définir un tableau de caractère. Le nombre maximum de caractères que comportera la chaîne sera égal au nombre d'éléments du tableau moins un (réservé au caractère de fin de chaîne.

char Nom_du_tableau[Nombre_d_elements]

Le nombre d'éléments que comporte le tableau définit la taille maximale de la chaine.

Astuce! En définissant le tableau de la manière suivante, vous mettez en évidence le nombre de caractère maximal de la chaîne:

char Nom_du_tableau[Nombre_d_elements + 1]

I. Les Chaînes de Caractères

Initialisation de chaine de caractères

Comme généralement en langage C, il faut initialiser la chaîne de caractère, c'est-à-dire remplir les cases du tableau avec des caractères, sachant que celui-ci devra obligatoirement contenir le caractère de fin de chaîne \0. Il y a deux façons de procéder:

- **Remplir manuellement le tableau case par case ;**
- **Utiliser les fonctions de manipulation de chaîne fournies dans les librairies standards.**

I. Les Chaînes de Caractères

Exemple: Initialisation manuelle

```
#include <stdio.h>
 void main()
{ char Chaine[20+1];
   Chaine[0]= 'B';
   Chaine[1]= 'o';
   Chaine[2]= 'n';
   Chaine[3]= 'j';
   Chaine[4]= 'o';
   Chaine[5]= 'u';
   Chaine[6]= 'r';
   Chaine[7]= '\0';
}
```

Une autre façon d'initialiser une chaîne de caractères:

```
#include <stdio.h>
void main(){
char Chaine[20+1]={ 'B', 'o', 'n', 'j', 'o', 'u', 'r', '\0' };
}
```

I. Les Chaînes de Caractères

Lecture et écriture de chaîne de caractères

Le langage C offre plusieurs possibilités de lecture ou d'écriture de chaînes:

- L'utilisation du code de format ***%s*** dans les fonctions ***printf*** et ***scanf,***
- Les fonctions spécifiques de lecture (***gets***) ou d'affichage (***puts***) d'une chaîne (une seule à la fois).

I. Les Chaînes de Caractères

Lecture et écriture de chaîne de caractères

Exemple:

```
int main()
  { char nom[20], prenom[20], ville[25] ;
    printf ("quelle est votre ville : ") ;
    gets (ville) ;
    printf ("donnez votre nom et votre prénom : ") ;
    scanf ("%s %s", nom, prenom) ;
    printf ("bonjour cher %s %s qui habitez à ", prenom, nom) ;
    puts (ville) ;
    getch();
}
```

I. Les Chaînes de Caractères

Lecture et écriture de de caractères

- La fonction getchar():

La fonction ***getchar()*** permet la récupération d'un seul caractère à partir du clavier.

La syntaxe d'utilisation est la suivante : ***var=getchar();*** ***var*** doit être de type ***char***.

Exemple :

```
#include <stdio.h>
 int main()
 {  char c;
    printf("Entrer un caractère:");
    c = getchar();
   printf("Le caractère entré est %c\n",c);
   return 0;
  }
```

I. Les Chaînes de Caractères

Fonctions d'entrée et de sortie standard pour caractère

- La fonction putchar()

La fonction putchar permet l'affichage d'un seul caractère sur l'´ecran de l'ordinateur. putchar constitue alors la fonction complémentaire de getchar.

La syntaxe d'utilisation est la suivante : ***putchar(var)***; où ***var*** est de type ***char.***

Exemple :

```
#include <stdio.h>
 int main()
{  char c;
   printf("Entrer un caractère:");
   c = getchar();
   putchar(c);
   return 0;
}
```

I. Les Chaînes de Caractères

Les fonctions de manipulation de chaînes de caractères

De nombreuses fonctions de manipulation de chaîne sont directement fournies. Ces fonctions se trouvent dans le fichier d'en-tête ***<string.h>,*** c'est la raison pour laquelle il faut ajouter la ligne suivante en début de programme: ***#include<string.h>***

Le fichier ***<string.h>*** contient les prototypes de nombreuses fonctions permettant de simplifier l'utilisation et la manipulation de chaînes (***environ une quarantaine***).

I. Les Chaînes de Caractères

Les fonctions de manipulation de chaînes de caractères

Cette bibliothèque propose des fonctions de maniement de chaînes de caractères

- Longueur d'une chaine

La fonction **strlen** fournit la longueur d'une chaîne de caractères. Le caractère nul de fin de chaîne n'est pas compté.

int strlen (const char * chaîne);

- ***strlen(ch)*** : renvoie la longueur d'une chaîne de caractère

I. Les Chaînes de Caractères

Les fonctions de manipulation de chaînes de caractères

- Comparaison de deux chaînes

La fonction **strcmp** compare une chaîne chaine1 avec une chaîne chaine2. la comparaison s'arrête quand deux caractères sont différents ou quand une chaîne est terminée. La fonction renvoie 0 si les chaînes sont identiques.

int strcmp (char * chaine1, char * chaine2);

La fonction **strcmp** compare deux chaînes en se limitant aux max premiers caractères.

int strcmp (char * chaine1, char * chaine2, int max);

I. Les Chaînes de Caractères

Les fonctions de manipulation de chaînes de caractères

- ***strcmp(ch1, ch2)*** compare deux chaînes et fournit une valeur entière positive si ch1>ch2, nulle si ch1=ch2 et négative si ch1<ch2.

- ***strncmp(ch1,ch2,lgmax)*** travaille comme ***strcmp*** mais elle limite la comparaison au nombre ***lgmax*** de caractères.

- ***stricmp(ch1, ch2)*** travaillent comme ***strcmp*** mais sans tenir compte de la différence entre majuscules et minuscules.

- ***- strnicmp(ch1, ch2, lgmax)*** travaillent comme ***strncmp*** mais sans tenir compte de la différence entre majuscules et minuscules.

I. Les Chaînes de Caractères

Les fonctions de manipulation de chaînes de caractères

- ***strchr(ch,caractère)*** recherche dans ch, la première position où apparait le caractère mentionné,
- ***strrchr(ch,caractère)*** opère de même que ***strchr*** mais en partant de la fin de ch,
- ***strstr(ch,ssch)*** recherche dans ch la première occurrence de la sous chaîne ssch,

I. Les Chaînes de Caractères

Les fonctions de manipulation de chaînes de caractères

- Concaténation de deux chaînes

La chaîne source peut être ajoutée à la fin de la chaîne destination avec la fonction **strcat**. La fonction renvoie l'adresse de la chaîne destination.

char * strcat (char * destination, char * source);

La fonction **strcat** ajoute au plus max caractères de la chaîne source à la fin de la chaîne destination.

char * strcat (char * destination, char * source, int max);

- *strcat(ch1,ch2*) concaténation de chaîne; recopie la seconde chaîne ch2 à la suite de la première ch1

I. Les Chaînes de Caractères

Les fonctions de manipulation de chaînes de caractères

- Copie d'une chaine

La fonction **strcpy** permet de copier une chaîne source dans une chaîne destination (y compris le caractère nul de fin de chaîne). La fonction renvoie l'adresse de la chaîne destination.

char * strcpy (char * destination, char * source);

La fonction **strcpy** copie au plus max caractères de la chaîne source dans la chaîne destination.

char * strcpy (char * destination, char * source, int max);

- ***strcpy(destin,source)*** recopie la chaîne source dans l'emplacement d'adresse destin.

I. Les chaînes de caractères

Définition et initialisation d'une chaîne

Une chaîne de caractères est une suite de caractères alphanumériques terminée par le caractère '\0'.

Les définitions suivantes **char texte[8]= ''bonjour''; ou char texte []= ''bonjour'';** permettent de réserver un tableau de 8 éléments dans lequel sont stockés les 7 caractères du mot «bonjour» suivis par le caractère nul '\0' (c'est-à-dire dont le code ASCII est 0).

Remarques :

- ne pas confondre les guillemets '' '' de la chaîne de caractères et les apostrophes ' ' qui entourent un caractère.
- Toute chaîne se termine automatiquement par '\0'

I. Les chaînes de caractères

Ecriture d'une chaîne

En considérant la chaîne comme un tableau, on peut réaliser l'affichage caractère par caractère jusqu'au moment où le caractère nul est atteint.

```
int i=0; char phrase []= ''il fait beau'';
while (phrase[i]) printf(''%c'', phrase[i++]);
```

On peut également utiliser **printf** avec le format **%s**.

On peut aussi utiliser la fonction **puts** qui n'affiche qu'une chaîne de caractère à la fois et se termine par une fin de ligne.

```
char phrase []= ''il fait beau'';
puts(phrase);
```

I. Les chaînes de caractères

Lecture d'une chaîne

La fonction **scanf** permet de lire les caractères avec le format **%s** mais la lecture s'effectue jusqu'à a rencontre d'un séparateur (espace, tabulation ou retour chariot). La chaine est complétée par le caractère '\0'.

```
char nom[20]; printf(''entrez le nom :'');
scanf(''%s'', nom);printf(''le nom est %s'', nom);
```

Si on désire saisir des chaînes avec des espaces ou des tabulations, il faut utiliser la fonction **gets**.

```
char nom[20]; printf(''entrez le nom :'');
gets(nom); printf(''le nom est %s'', nom);
```

I. Les chaînes de caractères

Tableaux de chaînes de caractères

- Pour traiter un ensemble de chaînes de caractères, on peut définir un tableau de chaînes, c'est-à-dire un tableau à deux dimensions de caractères.

Exemple : **char jours[7][9]={"lundi", "mardi", "mercredi", "jeudi", "vendredi", "samedi", "dimanche"};**

II. Rappel sur les tableaux numériques

II. Tableaux numériques

Rappel: Déclaration des tableaux

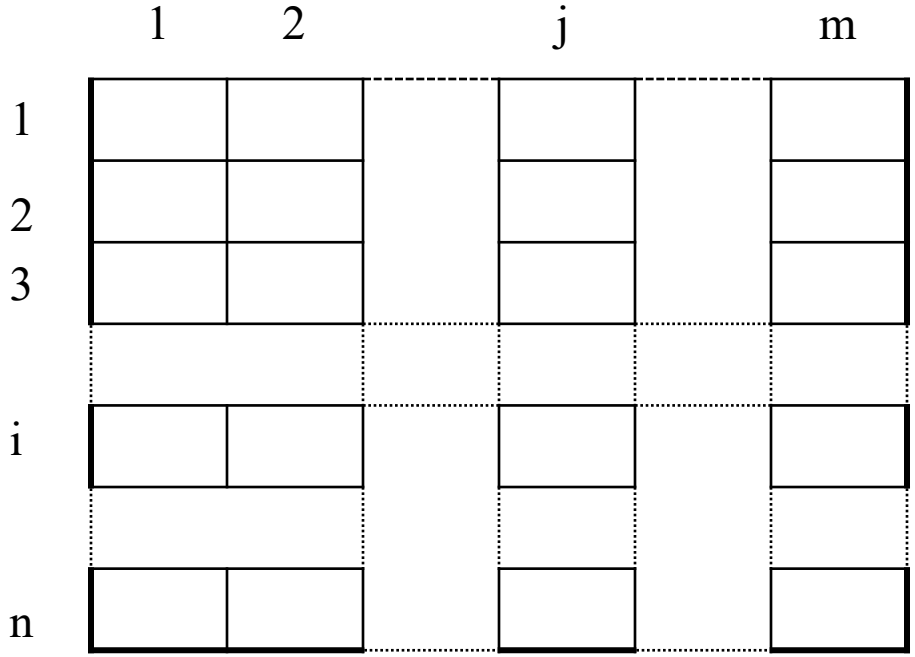
- Comme tout objet un tableau doit être déclaré avant tout utilisation
- 3 éléments fondamentaux définissent un tableau à un indice :
  - Son nom : identificateur respectant les règles classiques des identificateurs d'un programme
  - Le nombre de ses éléments
  - Le type de données qu'il contient

    T1[10]: entiers → tableau de 10 entiers maximum

    T2[ ]: réels → tableau de réels

II. Tableaux numériques

Rappel: Tableaux à plusieurs indices



II. Tableaux numériques

Rappel: Tableaux en C

♣Exemple :

```
int T1[13];
char T2[8][5][10];
float T3[6][15][9];
```

♣Principe :

<type><identificateur>[$taille_1$][$taille_2$]…[$taille_k$];

♣N'importe quelle référence à une case peut être utilisé comme une simple variable :

```
int i,j,k;
T1[i];
T2[i][j][k];
```

II. Tableaux numériques

Rappel: Tableaux en C

- La taille correspond au nombre de cases du tableau
- Attention : les indices, permettant de localiser le contenu d'une case d'un tableau, varient entre

0 et taille-1

- Il est possible d'affecter un tableau à un ensemble de valeurs dès sa déclaration par :

<type><identificateur>[taille$_1$][taille$_2$]…={val$_1$, val$_2$, …};

- Exemple :

int matrice[2][3]={1,2,3,4,5,6} $\Leftrightarrow$ $\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$

II. Tableaux numériques

Rappel: Exemple de programme sur les tableaux en C

```
#include <stdio.h>
int main()
{
    int i, somme, T[10];
    float moyenne;
    for (i=0;i<10;i++)
    {
            printf("T[%d]=",i);
            scanf("%d",&T[i]);
    }
    somme=0;
    for (i=0;i<=9;i++)
            somme=somme+T[i];
    moyenne=somme/10;
    printf("la moyenne des 10 valeurs est %f\n",moyenne);
 }
```

II. Tableaux numériques

Notion de tri

☞Un tableau T est dit « **trié en ordre croissant** » si tous les éléments consécutifs du tableau vérifient :

$$T[i-1] \leq T[i]$$

☞Il est admis qu'un

- ✓ tableau vide est trié
- ✓tableau ne contenant qu'un seul élément est trié

Ainsi :

- –Un tableau vide (n=0) est ordonné (trié),
- –Un tableau contenant un seul élément (n=1) est ordonné,
- –Un tableau T[1..n], n>1, est ordonné si

$$\forall\, i \in [2..n],\ T[i-1] \leq T[i]$$

II. Tableaux numériques
Notion de tri

☞Tri d'un tableau

- Soit un vecteur (tableau à une dimension) T[1..n] à valeurs dans un ensemble E de valeurs muni d'une relation d'ordre notée <
- Trier le vecteur T consiste à construire un vecteur **T1**[1..n] tel que :
  - **T1** soit trié,
  - **T1** et T contiennent les mêmes éléments.
- Le plus souvent **T** et **T1** sont le même vecteur ; **T1** est construit en permutant les éléments de **T** entre eux.

II. Tableaux numériques

Notion de tri

- On désigne par "**tri**" l'opération consistant à ordonner un ensemble d'éléments en fonction de *clés* sur lesquelles est définie une relation d'ordre.
- Les algorithmes de tri ont une grande importance pratique. Ils sont fondamentaux dans certains domaines, comme l'informatique de gestion où l'on tri de manière quasi-systématique des données avant de les utiliser.
- L'étude du tri est également intéressante en elle-même car il s'agit du domaine de l'algorithmique qui a été le plus étudié et qui a conduit à des résultats remarquables sur la construction d'algorithmes et l'étude de leur complexité.

II. Tableaux numériques

Les méthodes de tri

Tris stables	Tris non stables
Le tri par insertion	Le tri par sélection
Le tri bulle	Le tri à peigne
Le tri Shaker	Le tri Shell
Le tri Gnome	Le tri maximier
Le tri fusion	Le tri rapide

III. Algorithmes de tri stable